

Practical – Week11

1. Develop a multithreaded counter application using POSIX threads (pthread_create, pthread_join). Demonstrate race conditions by allowing multiple threads to update a shared variable concurrently.

```
GNU nano 8.7.1                                counter.c *
#include <stdio.h>
#include <pthread.h>

#define THREADS 4
#define COUNT 1000000

int counter = 0;  // Shared variable

void *increment(void *arg) {
    for (int i = 0; i < COUNT; i++) {
        counter++;
    }
    return NULL;
}

int main() {
    pthread_t t[THREADS];

    // Create threads
    for (int i = 0; i < THREADS; i++) {
        pthread_create(&t[i], NULL, increment, NULL);
    }

    // Wait for all threads
    for (int i = 0; i < THREADS; i++) {
        pthread_join(t[i], NULL);
    }

    printf("Expected counter: %d\n", THREADS * COUNT);
    printf("Actual counter: %d\n", counter);

    return 0;
}
```

```
ajithmutyala@Ajith:~$ nano counter.c
ajithmutyala@Ajith:~$ gcc -pthread -O0 counter.c -o counter
ajithmutyala@Ajith:~$ ./counter
Expected counter: 400000000
Actual counter: 14198641
ajithmutyala@Ajith:~$ ./counter
./counter
./counter
Expected counter: 400000000
Actual counter: 11506657
Expected counter: 400000000
Actual counter: 11401231
Expected counter: 400000000
Actual counter: 12499124
ajithmutyala@Ajith:~$ |
```

Report: The multithreaded counter was implemented using `pthread_create()` and `pthread_join()`. Concurrent updates to the shared variable caused a race condition, resulting in a difference between the expected and actual counter values.

2. Modify the previous program using mutex locks (`pthread_mutex_lock`, `pthread_mutex_unlock`) and compare the outputs with the race-condition version. Analyze the impact of synchronization.

```

GNU nano 8.7.1                                counter_mutex.c *
#include <stdio.h>
#include <pthread.h>

#define THREADS 4
#define COUNT 1000000

int counter = 0;
pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;
void *increment(void *arg) {
    for (int i = 0; i < COUNT; i++) {
        pthread_mutex_lock(&lock);
        counter++;
        pthread_mutex_unlock(&lock);
    }
    return NULL;
}
int main() {
    pthread_t t[THREADS];
    for (int i = 0; i < THREADS; i++) {
        pthread_create(&t[i], NULL, increment, NULL);
    }
    for (int i = 0; i < THREADS; i++) {
        pthread_join(t[i], NULL);
    }
    printf("Expected counter: %d\n", THREADS * COUNT);
    printf("Actual counter: %d\n", counter);
    pthread_mutex_destroy(&lock);
    return 0;
}

```

```

ajithmutyala@Ajith:~$ nano counter_mutex.c
ajithmutyala@Ajith:~$ gcc -pthread -O0 counter_mutex.c -o counter_mutex
ajithmutyala@Ajith:~$ ./counter_mutex
Expected counter: 40000000
Actual counter: 40000000
ajithmutyala@Ajith:~$ ./counter_mutex
./counter_mutex
./counter_mutex
Expected counter: 40000000
Actual counter: 40000000
Expected counter: 40000000
Actual counter: 40000000
Expected counter: 40000000
Actual counter: 40000000
ajithmutyala@Ajith:~$ ./counter
Expected counter: 40000000
Actual counter: 11173720
ajithmutyala@Ajith:~$ |

```

```
ajithmutyala@Ajith:~$ ./counter
Expected counter: 400000000
Actual counter: 11253920
ajithmutyala@Ajith:~$ ./counter_mutex
Expected counter: 400000000
Actual counter: 400000000
```

Report: The program used `pthread_mutex_lock()` and `pthread_mutex_unlock()` to prevent race conditions. The `./counter` and `./counter_mutex` outputs showed that mutex locks ensure correct counter updates, but increase execution time.